DSY1107 · EVALUACIÓN PARCIAL N°2

Pedidos360

Aplicación cloud-native: EC2 · Spring Boot · GitHub Actions · Cognito · API Gateway · React · RabbitMQ

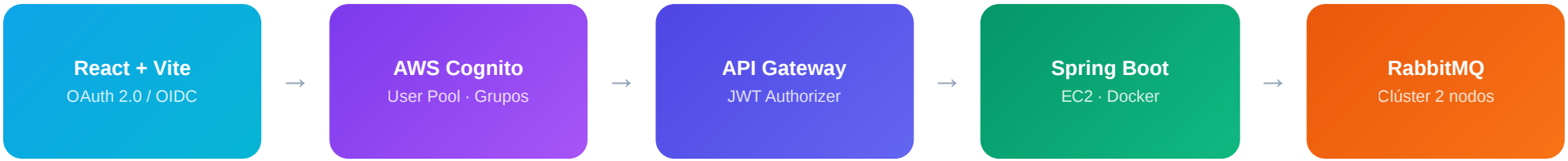
AWS EC2 | Spring Boot 3 | CI/CD GHCR | Cognito OIDC | API Gateway | RabbitMQ 2 nodos

Presentación individual · 5–10 minutos

2026

1 Arquitectura de la solución

VISIÓN GENERAL



FLUJO DE AUTENTICACIÓN

- El frontend redirige al **Hosted UI** de Cognito (code + PKCE).
- Cognito devuelve un **JWT** con el claim `cognito:groups`.
- El API Gateway valida el token antes de enrutar.
- Spring Security valida firma, issuer, expiración y audience.

FLUJO DE MENSAJERÍA

- **Direct** exchange → notificaciones.
- **Topic** exchange → tickets y documentos.
- Error de negocio → **NACK** → DLX → **DLQ**.
- Topología declarada por código y por `definitions.json`.

2 Tarea 1 · Instancia EC2 y acceso SSH

INFRAESTRUCTURA

PROCEDIMIENTO

- Key pair `pedidos360-key.pem` y Security Group con puertos 22, 80, 8080, 15672, 15673.
- Instancia **Amazon Linux 2023** `t3.micro`, etiquetada `pedidos360-ec2`.
- Bootstrap con `infra/ec2-setup.sh`: instala Docker Engine y Compose v2.
- Comprobación de acceso por SSH con hostname y versión del sistema.

Comando de verificación

```
ssh -i pedidos360-key.pem ec2-user@<IP> "hostname && cat /etc/os-release | head -2"
```

Resultado esperado

ip-172-31-xx-xx.ec2.internal
NAME="Amazon Linux"
VERSION="2023"

2
NODOS RMQ

3
COLAS + DLQ

3 Tarea 2 · Spring Boot, puerto y GitHub

BACKEND

STACK Y ESTRUCTURA

- **Spring Boot 3.3.5** · Java 17 · Maven.
- Web, Security, OAuth2 Resource Server, AMQP, Validation, Actuator.
- Paquetes: `config`, `messaging`, `service`, `web`, `dto`, `exception`.
- Puerto parametrizado: `server.port: ${SERVER_PORT:8080}`.

Compilación

`mvn -B clean verify` → **BUILD SUCCESS**

SEPARACIÓN DE RESPONSABILIDADES

CAPA	RESPONSABILIDAD
`web`	REST + validación
`service`	Reglas de negocio
`messaging`	Publisher + consumers
`config`	Topología RabbitMQ y seguridad

El código está versionado en GitHub con `main` como rama de despliegue.

4 Tarea 3 · GitHub Actions + GHCR + Docker → EC2

CI/CD

1 · Build & Test
Maven con JDK 17, publica el JAR como artefacto.

2 · Docker → GHCR
Buildx publica `ghcr.io/<owner>/pedidos360-backend` con tags `latest` y `sha`.

3 · Deploy EC2
SSH: `docker pull`, recrea el contenedor en `-p 80:8080`.

SECRETS UTILIZADOS

- `EC2_HOST`, `EC2_USER`, `EC2_SSH_KEY`
- `GHCR_PAT` para el pull desde la instancia
- `COGNITO_ISSUER_URI`, `RABBITMQ_*`, `CORS_ALLOWED_ORIGINS`

VERIFICACIÓN

`docker ps --filter name=pedidos360-backend`
`curl http://<IP>/api/health` → **200 UP**

5 Tarea 4 · Cognito y API Gateway

SEGURIDAD

IDAAS · AWS COGNITO

- User Pool `pedidos360-pool` con login por email.
- Grupos: **ADMIN** **COLLABORATOR** **CUSTOMER**
- App Client SPA sin secret · Authorization Code + PKCE.
- Hosted UI con dominio propio de Cognito.

API GATEWAY + AUTORIZACIÓN

- HTTP API con integración proxy a la EC2.
- Autorizador **JWT** (issuer + audience del User Pool).
- Ruta pública: `GET /api/health`.
- Rutas protegidas: `ANY /{proxy+}`.

Reglas del backend por rol:

GET autenticado · **POST/PUT** ADMIN/COLLAB · **DELETE** ADMIN

6 Tarea 5 · React + Cognito (login probado)

FRONTEND

IMPLEMENTACIÓN

- React 18 + TypeScript + Vite.
- `react-oidc-context` + `oidc-client-ts`.
- `response_type: 'code'`, scope `openid email profile`.
- El token se adjunta como `Authorization: Bearer`.

DEMO

1. Clic en **Iniciar sesión con Cognito**.
2. Hosted UI → `admin@pedidos360.cl`.
3. La app muestra email, grupo **ADMIN** y el JWT.
4. Crear pedido → aparece en la tabla.

El grupo determina las acciones disponibles; un CUSTOMER solo puede leer.

7 RabbitMQ · Topología de 3 colas + DLQ

RÚBRICA 12 % · 13 %

EXCHANGES Y BINDINGS

EXCHANGE	TIPO	ROUTING KEY	COLA
pedidos.direct	direct	pedido.creado	notificaciones.queue
pedidos.topic	topic	pedido.*.ticket	tickets.queue
pedidos.topic	topic	pedido.*.documento	documentos.queue
pedidos.dlx	direct	<dominio>.dlq	<dominio>.dlq

CENTRALIZACIÓN

Todos los nombres en `application.yml` bajo `app.rabbit`.
Leídos con `@ConfigurationProperties` en `RabbitProperties`.
Sin nombres hardcodeados en consumidores.

3 COLAS

3 DLQ

3 EXCHANGES

8 ACK/NACK, DLQ y microservicio de administración

RÚBRICA 15 % · 20 % · 13 % · 10 % · 7 %

CONSUMIDORES POR DOMINIO

- `NotificacionConsumer` → notificaciones.queue
- `TicketConsumer` → tickets.queue (cantidad > 100 falla)
- `DocumentoConsumer` → documentos.queue

`basicAck(tag,false)` al procesar OK.
`basicNack(tag,false,false)` ante error → DLQ.
Retry: 3 intentos con backoff exponencial.

API DE ADMINISTRACIÓN

MÉTODO	RUTA
GET	/api/admin/queues
POST	/api/admin/queues
DELETE	/api/admin/queues/{name}
GET / POST	/api/admin/exchanges

Lógica en `RabbitAdminService` · parámetros validados con `@NotBlank` y `@Pattern`.

9 Clúster RabbitMQ de 2 nodos

DOCKER-COMPOSE

SERVICIOS

rabbitmq-1 · nodo semilla

AMQP `:5672` · Management `:15672`
Importa `definitions.json` (3 colas + 3 DLQ + exchanges).

rabbitmq-2 · se une al clúster

AMQP `:5673` · Management `:15673`
`rabbitmqctl join_cluster rabbit@rabbitmq-1`

VERIFICACIÓN EN VIVO

`docker exec rabbitmq-1 rabbitmqctl cluster_status`
→ rabbit@rabbitmq-1, rabbit@rabbitmq-2
`docker exec rabbitmq-1 rabbitmqctl list_queues name messages`
→ 3 colas + 3 DLQ

Cookie Erlang compartida y política `ha-all` para replicación.

10 Cumplimiento de la rúbrica

100 %

INDICADOR	%	EVIDENCIA	ESTADO
Nombres centralizados en application.yml	12 %	`application.yml:39-59` + `RabbitProperties`	✔
Beans Queue / Exchange / Binding	13 %	`RabbitMQConfig.java:33-131`	✔
Separación negocio / mensajería	10 %	`PedidoService` ↔ `PedidoEventPublisher`	✔
Consumers @RabbitListener por dominio	15 %	3 consumidores (notif/tickets/docs)	✔
ACK/NACK + retry + DLQ	20 %	`basicAck` / `basicNack(false)` + retry	✔
Microservicio admin REST	13 %	`RabbitAdminController`	✔
Lógica encapsulada en servicio	10 %	`RabbitAdminService`	✔
Validación de parámetros	7 %	`@NotBlank` + `@Pattern` + handler	✔

Además: 2 nodos en clúster, 3 colas con DLQ, exchanges direct y topic, y docker-compose operativo.